

CHAPTER 11

跳躍與地面檢測 (JUMP LOGIC)

複習：上週重點

- [x] 讀取左右輸入 (`Input.GetAxisRaw`).
- [x] 物理移動 (`rb.velocity`).
- [x] 角色翻面 (`localScale`).

現在主角可以在地上滑來滑去了...

但這可是橫向捲軸遊戲 (Platformer) !

不會跳躍，就只是個平面遊戲。

本章目標

1. 實作跳躍邏輯 (`Jump Force`).
2. 理解輸入差異 (`GetButtonDown`).
3. **核心難點：地面檢測** (Ground Check).
4. 防止無限二段跳 (Flappy Bird 問題).

跳躍原理 (JUMP FORMULA)

跳躍其實就是一個瞬間向上的力，或者瞬間向上的速度。

不同於左右移動 (需要持續設值)，跳躍是**瞬間** (Impulse) 的。

```
// 保持原本的 x 速度 (不然跳起來會定桿)  
// Y 軸設為跳躍力道  
rb.velocity = new Vector2(rb.velocity.x, jumpForce);
```

輸入偵測：按下瞬間

我們不能用 `Input.GetKey` (按住)，不然主角會像火箭一樣飛上去。

我們要用 `Input.GetButtonDown` (按下瞬間)。

- 按鍵設定：Unity 預設的 `Jump` 對應 空白鍵 (Space)。

```
void Update()  
{  
    // 當按下空白鍵的 "那一幀"  
    if (Input.GetButtonDown("Jump"))  
    {  
        Jump();  
    }  
}
```

實作測試 1：無限跳躍

1. 在 `PlayerController` 加入 `jumpForce` 變數。
2. 寫好 `Jump` 方法。
3. Play。
4. 按一下空白鍵 -> 跳起來了！
5. 在空中連按空白鍵 -> **飛起來了！(Flappy Bird)**

這不是我們要的，我們需要「踩到地板」才能跳。

核心難題：怎麼知道踩到地板？

電腦不知道什麼是地板。

即使用了 Collage 碰撞，電腦也分不清楚是「踩到地板」還是「頭撞到天花板」或是「身體貼著牆」。

我們需要一個專門的感應器。

地面檢測方案

方案 A：碰撞事件 (ONCOLLISIONENTER)

- 當撞到東西時設為 true。
- 缺點：如果臉貼牆壁，也算撞到，會變成可以爬牆跳。

方案 B：射線檢測 (RAYCAST)

- 從腳底射出一條雷射光。
- 缺點：只有一條線，如果站在懸崖邊緣可能會射空。

方案 C：圓形檢測 (OVERLAPCIRCLE) 🏰

- 在腳底畫一個小圓圈。
- 檢查圓圈內有沒有「地板圖層」的東西。
- 最穩定的做法！

步驟 1：設定 LAYER

我們要告訴電腦「什麼是地板」。

1. Inspector 右上角 Layer -> Add Layer。
2. 新增 Layer 6: **Ground**。
3. 選取場景中的 Grid/Tilemap (Ground) 物件。
4. 將 Layer 改為 **Ground**。
 - (如果有子物件，記得選 **Yes, change children**)。

步驟 2：設定感應點 (GROUND CHECK)

我們需要知道腳底在哪裡。

1. 在 Hierarchy 的 Player 下方按右鍵 -> **Create Empty**。
2. 命名為 **GroundCheck**。
3. 將它移動到主角的腳底板正中心。
4. (選用) 點選 Inspector 左上角的方塊圖示，給它一個顏色標記，方便觀察。

步驟 3：撰寫程式 (變數)

在 `PlayerController` 加入變數：

```
[Header("Ground Check Settings")]  
public Transform groundCheck;    // 感應點的位置  
public float groundCheckRadius = 0.2f; // 感應圓圈半徑  
public LayerMask groundLayer;    // 地板是哪個圖層  
public bool isGrounded;          // 目前是否在地板上 (除錯用)
```

步驟 4：撰寫程式 (邏輯)

在 `Update()` 或 `FixedUpdate()` 中進行檢測：

```
void Update()
{
    // 1. 發射圓形感應
    // Physics2D.OverlapCircle(圓心, 半徑, 指定圖層)
    // 如果有碰到東西，回傳 true
    isGrounded = Physics2D.OverlapCircle(groundCheck.position, groundCheckRadius, groundLayer);

    // 2. 跳躍判斷
    // 必須按下按鍵 且 在地板上
    if (Input.GetButtonDown("Jump") && isGrounded)
    {
        Jump();
    }
}
```

步驟 5：視覺化除錯 (GIZMOS)

雖然程式寫好了，但在 Scene 視窗看不到那個圓圈。
加入這個神奇方法：

```
// 只有在 Scene 視窗會執行的繪圖方法
void OnDrawGizmosSelected()
{
    if (groundCheck != null)
    {
        Gizmos.color = Color.red;
        // 畫出空心圓
        Gizmos.DrawWireSphere(groundCheck.position, groundCheckRadius);
    }
}
```

步驟 6：設定參數

1. 回到 Unity，選取主角。
2. **Ground Check**：把子物件 **GroundCheck** 拖進去。
3. **Ground Layer**：下拉選單勾選 **Ground**。
4. **Radius**：預設 0.2 應該剛好。

最終測試

1. 按下 Play。
2. 看 Inspector 的 `Is Grounded` 勾勾。
 - 站在地上時 -> 打勾。
 - 跳起來時 -> 取消。
3. 試著連按空白鍵 -> **空中不能跳了！**
4. 試著站在懸崖邊緣 -> 只要感應圓圈還有一點點碰到地，就能跳。

手感調整 (GAME FEEL)

如果不滿意跳躍手感：

1. 太飄 (Moon Gravity)：

- 增加 Rigidbody 的 **Gravity Scale** (例如改到 3~4)。
- 相對地，**Jump Force** 也要加大 (例如改到 12~15)。
- 這是瑪利歐類遊戲的秘訣 (高重力 = 快速落地 = 節奏快)。

2. 跳太低：增加 Jump Force。

進階挑戰：二段跳 (DOUBLE JUMP)

很多遊戲都有二段跳。

邏輯提示：

1. 多一個變數 `bool canDoubleJump`。
2. 如果 `isGrounded` 為 true -> `canDoubleJump = true`。
3. 如果按下跳躍：
 - 在地板上 -> 跳。
 - 在空中 且 `canDoubleJump` -> 跳，並把 `canDoubleJump = false`。

常見錯誤 (DEBUG)

Q: `isGrounded` 永遠是 false ?

A:

1. 檢查 Ground Layer 有沒有選對 (`Ground`) 。
2. 檢查 Tilemap 物件的 Layer 有沒有設成 `Ground` 。
3. 檢查 GroundCheck 的位置是不是浮在半空中 (太高) 。

Q: `isGrounded` 永遠是 true ?

A:

1. GroundCheck 位置太低，插到地板裡面了？
2. Ground Layer 不小心勾到了 Player 自己 (偵測到自己的腳) 。

總結

今天我們完成了平台遊戲最核心的機制：

1. **瞬間力道**：`velocity` 改寫 Y 軸。
2. **圖層遮罩**：`LayerMask` 過濾碰撞物。
3. **區域檢測**：`OverlapCircle` 判斷接地。

現在主角能跑能跳，遊戲性已經出來了！

下週預告

主角雖然動得很好，但看起來像個殭屍 (完全沒動作)。

下週我們要把美術圖換成動畫：

- **Animator** (動畫控制器)。
- **Animation Clips** (跑步、跳躍、閒置)。
- 讓程式控制動畫切換。

Q & A

- 為什麼有時候跳躍有延遲？
 - 檢查 `GetButtonDown` 是不是寫在 `FixedUpdate` 了？(輸入一定要在 Update 抓)。
- 為什麼從很高的地方掉下來會穿過地板？
 - 速度太快了。請將 Collision Detection 改為 Continuous。

(助教巡堂協助)